TECH REFERENCE

# Knowledge Distillation · Recommendation Reason Generation

Technical Reference: Temperature Scaling, FD-TVS, Grounding, LLM Safety

Author 1[1], Author 2[1]

[1]*Organization Name*
*contact@org.com*

**Scope.**

- Knowledge distillation: PLE Teacher → LGBM Student (Temperature Scaling, custom objective, calibration)
- Feature selection: LGBM gain importance, 1205D → 280D per task (17 feature groups, 2026-04-28)
- FD-TVS scoring: dynamic (segment × behavior) task weights
- Reason generation: L1 Template → L2a LLM rewrite → L2b 3-Agent review + SelfChecker
- Layer split: 7 distilled LGBM + 3 direct LGBM + 3 Layer-3 rule-based
- Serving: Lambda Container Image + LanceDB (recommendation cases) + DynamoDB (cache / audit)

# Contents

**Design vs. Implementation Dimensionality Notice**

This document is written based on the **full-bank design (734D)**. The current Santander benchmark implementation uses **1205D input (17 feature groups, 2026-04-28)**. Dimensional figures in the body (734D, 200D, 140D, etc.) reflect the full-bank design; intermediate dimensions may differ in the actual Santander implementation. For the dimension specifications of the actual implementation, refer to `outputs/phase0/feature_schema.json`.

# Knowledge Distillation

## Design Philosophy and Motivation

### Core Problem

The PLE Teacher model used in the AWS Santander benchmark has approximately 2.6M parameters and is trained on g4dn.xlarge (8GB T4 VRAM) with an inference latency of roughly 50ms per batch of 1,024 samples on GPU. To eliminate the cost of running a GPU instance continuously for daily batch inference over millions of customers, and to meet the real-time recommendation SLA (under 10ms) on CPU-only Lambda, we distill the teacher into a LightGBM student.

### Solution Strategy

Through Knowledge Distillation (Hinton et al., 2015), the *implicit knowledge (dark knowledge)* encoded in the Teacher's output distribution is transferred to a LightGBM Student. The Student achieves approximately 5ms per 1K batch on an 8GB RAM CPU, delivering a 10x speed improvement while keeping performance degradation within 3 percentage points.

### Teacher–Student Comparison

| Property | PLE Teacher | LGBM Student |
|---|---|---|
| Model Architecture | PLE-adaTT + Cluster Head | LightGBM (per-task independent) |
| Parameters | ~2.6M (AWS Santander) | ~500 trees/task |
| Memory | ~8GB VRAM (g4dn.xlarge GPU) | ~1GB RAM (Lambda CPU) |
| Inference Speed | ~50ms / 1K batch | ~5ms / 1K batch |
| Feature Dimensionality | 734D (design) / 1205D (Santander 17-group impl.) | 200D (after IG selection, design basis) |
| Training Data | Raw features + labels | Raw features + Hard Labels + Soft Labels |

*Cross-architecture distillation* (DNN → GBDT) is possible because Knowledge Distillation transfers knowledge through output distributions, not parameters. "Train with deep learning, serve with GBDT" is the de facto standard in recommendation, finance, and advertising domains (Borisov et al., NeurIPS 2022).

# Temperature Scaling

### Mathematical Definition

A Temperature parameter $T$ is introduced into the standard softmax.

$$p_i^T = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)} \tag{1}$$

- $T = 1$: Standard softmax. Probability concentrates on the maximum logit.
- $T = 5$ (Default): Smoothed distribution that reveals relative relationships between classes.
- $T \to \infty$: Uniform distribution. Information is lost.

### Correspondence with the Boltzmann Distribution

This formula is mathematically isomorphic to the Boltzmann distribution from statistical mechanics, $P(E_i) = \exp(-E_i/(k_B T))/Z$. The logit $z_i$ corresponds to (negative) energy and $T$ to absolute temperature; both fields share the same exponential family distribution.

### Temperature Range Configuration

$T$ is constrained to $[3, 7]$.

- $T = 3$: Binary tasks (CTR, CVR, Churn, Retention)
- $T = 5$: Default
- $T = 7$: Multi-class tasks (NBA 12-class, Timing 28-class)
- $T > 10$: Risk of excessive information loss

> **LGBM Student Temperature**
> For LGBM students, $T = 1$ (standard softmax) is used when the teacher outputs are passed as soft labels to the GBDT objective. The LGBM custom objective receives probability vectors directly — applying $T > 1$ smoothing on top of LGBM's own internal optimization is redundant and can degrade calibration. The $T = 3$–$5$ range is appropriate for neural-to-neural distillation, not DNN→GBDT.

# Dark Knowledge: Information Content of Soft Labels

### Limitations of Hard Labels

Hard labels (one-hot) encode only $\log_2(C)$ bits of information. For NBA 12-class, this is approximately 3.6 bits, and no relational information between secondary class choices is captured at all.

### Rich Structure of Soft Labels

The Teacher's softmax output contains $(C-1)$ continuous probability values, providing far more information than $\log_2(C)$ bits.

$$p_{\text{teacher}} = [0.72, 0.14, 0.08, 0.03, 0.01, ...] \tag{2}$$

From this distribution, one can read the primary prediction (72%), the secondary choice structure (B 14% > C 8%), inter-class similarity, and the level of uncertainty. This is what Hinton termed *Dark Knowledge*.

**Label Smoothing Effect**

Training with soft labels has a regularization effect analogous to label smoothing. Whereas hard labels push the model toward extreme probabilities of 0 or 1 (inducing overfitting), soft labels encourage the model to produce reasonable probability distributions, thereby promoting generalization.

## Unified Distillation Loss

**Unified Loss Function**

$$\mathcal{L}_{\text{distill}} = \alpha \cdot \mathcal{L}_{\text{hard}} + (1-\alpha) \cdot T^2 \cdot \mathcal{L}_{\text{soft}} \tag{3}$$

- $\alpha$: Hard/Soft mixing ratio (Default 0.3, i.e., 30% ground truth + 70% Teacher opinion)
- $T^2$: Gradient magnitude correction scaling

**Mathematical Derivation of the $T^2$ Correction**

From the chain rule, $\partial\hat{y}/\partial z = (1/T)\sigma(z/T)(1-\sigma(z/T))$. The $1/T$ factor accumulates, reducing the total gradient by $1/T^2$. Multiplying by $T^2$ restores the original scale. When $T = 5$, the gradient is reduced to $1/25$, so multiplying by $T^2 = 25$ compensates.

**Per-Task Loss Functions**

**Binary classification (CTR, CVR, Churn, Retention):**

$$\mathcal{L}_{\text{binary}} = \alpha \cdot \text{BCE}(\hat{y}, y) + (1-\alpha) \cdot T^2 \cdot D_{\text{KL}(p_t \parallel p_s)} \tag{4}$$

where $p_t = \sigma(z_t/T)$ and $p_s = \sigma(z_s/T)$.

**Multi-class (NBA 12-class, Life-stage 6-class, Timing 28-class):**

$$\mathcal{L}_{\text{multi}} = \alpha \cdot \text{CE}(z_s, y) + (1-\alpha) \cdot T^2 \cdot D_{\text{KL}(\text{softmax}(z_t/T) \parallel \text{softmax}(z_s/T))} \tag{5}$$

**Regression (LTV, Engagement):**

$$\mathcal{L}_{\text{reg}} = \alpha \cdot \text{MSE}(\hat{y}_s, y) + (1-\alpha) \cdot \text{MSE}(\hat{y}_s, \hat{y}_t) \tag{6}$$

Temperature Scaling is not meaningful for regression, so the $T^2$ correction is not applied.

**KL-Divergence**

$$D_{\text{KL}(q \parallel p)} = \sum_i q_i \log \frac{q_i}{p_i} = \underbrace{-H(q)}_{\text{constant}} + \underbrace{H(q,p)}_{\text{cross-entropy}} \tag{7}$$

Forward KL $D_{\text{KL}(\text{Teacher} \parallel \text{Student})}$ is used. Its mean-seeking characteristic forces the Student to cover all important modes of the Teacher. Reverse KL has mode-seeking characteristics and risks ignoring certain modes, making it unsuitable for distillation.

# LGBM Feature Importance-Based Feature Selection

Feature selection is performed using the **gain importance** of the trained LGBM Student model. Teacher model IG (Integrated Gradients) attribution is not used — the teacher has already been distilled, and selecting features from the serving model's (LGBM Student's) perspective aligns with the design intent.

### Design Rationale

**Serving model alignment**: The deployed model is the LGBM Student. Gain importance reflects what features LGBM actually computes. Teacher (PLE) IG attributions can differ substantially from Student importance due to the architectural gap between deep networks and gradient-boosted trees.

**Operational stability**: LGBM gain computation is a fast post-hoc step on the trained Student. Teacher IG requires backpropagation through the full PLE graph at production scale (941K customers, 1205D features), causing out-of-memory failures.

**Interpretability alignment**: SHAP/gain explanations derived from the LGBM Student are directly grounded in the model that generated the recommendation, satisfying EU AI Act Art. 13 (explanations must reflect the actual decision mechanism).

### 2-Stage Pipeline

*The dimensionality figures below are based on the full-bank design (734D). The Santander 17-group implementation uses 1205D input.*

**Full-bank design basis:** 734D (design) / 1205D (Santander 17-group impl.) → 140D (Stage 1) → final LGBM input

**Stage 1 – LGBM Gain Importance Filter:** Top-$k$ features capturing approximately 95% of cumulative gain importance are retained per task. The resulting feature set is typically 40–80 features per task (down from the full input dimension).

**Stage 2 – Mandatory Feature Preservation:** Seven features always included regardless of LGBM importance:

- TDA: `persistence_entropy`, `landscape_peak`
- Economics: `mpc`, `income_elasticity`, `permanent_income_ratio`
- FinEng: `sharpe_ratio`, `volatility`

# LightGBM Custom Objective

### Implementation Structure

`DistillationLossNumpy` provides gradient/hessian to LightGBM's `fobj`.

$$\text{grad} = \alpha \cdot \text{grad}_{\text{hard}} + (1-\alpha) \cdot T^2 \cdot \text{grad}_{\text{soft}} \tag{8}$$

$$\text{hess} = \alpha \cdot \text{hess}_{\text{hard}} + (1-\alpha) \cdot T^2 \cdot \text{hess}_{\text{soft}} \tag{9}$$

### Soft Label Delivery Technique

Since a LightGBM Dataset supports only `label` and `weight`, hard labels are passed via `get_label()` and soft labels via `get_weight()`.

**Distillation Performance Comparison**

| Method | CTR AUC | NBA Accuracy | LTV RMSE |
|---|---|---|---|
| LGBM (Hard Label only) | 0.812 | 0.634 | 1.247 |
| LGBM (Distilled, $T = 5$) | 0.841 | 0.698 | 1.089 |
| PLE Teacher (original) | 0.856 | 0.723 | 1.021 |

## 10-Stage DAG Orchestration

The full distillation pipeline is executed as a 10-stage DAG in `distillation_entrypoint.py`.

| Stage | Content |
|---|---|
| 1 | Teacher inference (logit generation over the full dataset) |
| 2 | Soft label generation (Temperature Scaling applied) |
| 3 | LGBM gain importance feature selection (1205D → 140D per task) |
| 4 | Student training (per-task independent LGBM) |
| 5 | Validation (5 criteria: AUC, RMSE, Accuracy, etc.; metric aggregation is task-type-specific — binary tasks use AUC, regression tasks use RMSE, multi-class tasks use top-k Accuracy) |
| 6 | MLflow model registry registration |
| 7 | ONNX conversion (including ZipMap removal) |
| 8 | Triton packaging (config.pbtxt generation) |
| 9 | Integration validation (ONNX-PyTorch numerical equivalence) |
| 10 | Deployment artifact upload |

## Adaptive Distillation Threshold

The distilled LGBM is not accepted unconditionally. Acceptance requires meeting an **adaptive threshold** of at least 2× the random-baseline AUC per task. Below this floor, the student is classified as SKIP — no worse than random, but the distillation round is discarded and the teacher's direct inference path is activated instead.

| Task Type | Floor Threshold | Policy on SKIP |
|---|---|---|
| Binary (CTR/CVR/Churn) | AUC $> 0.5 + \delta_{\text{min}}$ (2× random) | Discard student; activate Layer 2 (teacher direct) |
| Multi-class (NBA) | Top-1 Accuracy $> 1/\text{C} + \delta_{\text{min}}$ | Discard student; fall back to popularity rule |

| | | |
|---|---|---|
| Regression (LTV) | RMSE $<$ baseline mean predictor | Discard student; use global mean |

This prevents deploying a distilled model that is worse than naive baselines, aligning with the guideline's Governance principle §1.4 (pre-launch risk-mitigation verification).

## Calibration: Platt Scaling

FD-TVS Stage 1 requires all task probabilities to be on a common $[0, 1]$ scale. Probability-critical tasks (churn, product_stability, cross_sell_count) apply **Platt scaling** post-distillation.

$$p_{\text{calibrated}} = \sigma(a \cdot f(\boldsymbol{x}) + b) \tag{10}$$

where $f(\boldsymbol{x})$ is the raw LGBM score, and $a, b$ are fitted on a held-out calibration set (never on training data). Calibration reduces Expected Calibration Error (ECE) and ensures that a predicted probability of 0.8 corresponds to an empirical positive rate of approximately 80%.

## Temporal Drift Monitoring

Beyond population-level PSI, three metrics monitor distributional shift over time for the distilled LGBM's input features and prediction outputs:

| Metric | What It Detects | Threshold |
|---|---|---|
| PSI (Population Stability Index) | Overall input distribution shift | $> 0.25$ critical |
| JSD (Jensen-Shannon Divergence) | Symmetric distribution distance; more stable for near-zero cells than KL | $> 0.10$ alert |
| Rank Correlation (Spearman) | Feature importance rank stability across retraining cycles | $< 0.70$ alert |

All three are computed daily per-feature. The `ConsecutiveDriftTracker` triggers automatic retraining when PSI $> 0.25$ persists for 3 consecutive days. JSD and rank correlation alerts are forwarded to the OpsAgent for narrative analysis.

# FD-TVS Scoring Engine

## Design Philosophy

FD-TVS (Financial DNA-based Target Value Score) is a 4-stage composite scoring engine that combines model predictions with customer-level business context. The core design principle is **multiplicative combination**: if any single factor approaches zero, the overall score is vetoed, enforcing a "risk-first" principle.

A simple summation $\sum p_i$ fails to distinguish a window shopper (CTR=0.9, CVR=0.1) from an actual buyer (CTR=0.5, CVR=0.5) — both sum to 1.0. The multiplicative structure grants each dimension veto power.

## Master Formula

$$\text{FD-TVS} = \underbrace{S_{\text{task}}}_{\text{What}} \times \underbrace{W_{\text{DNA}}}_{\text{Who}} \times \underbrace{V_{\text{TDA}}}_{\text{When}} \times \underbrace{(1-R)}_{\text{Safe?}} \times \underbrace{f \cdot e}_{\text{Appropriate?}} \tag{11}$$

| Component | Description | Range |
|---|---|---|
| $S_{\text{task}}$ | Task Weighted Sum | $[0, 1]$ |
| $W_{\text{DNA}}$ | Financial DNA Modifier | $\{0.8, 1.0, 1.2\}$ |
| $V_{\text{TDA}}$ | Behavioral Velocity | $[1.0, 1.15]$ |
| $R$ | Risk Penalty | $[0, 1]$ |
| $f$ | Fatigue Decay | $[0, 1]$ |
| $e$ | Engagement Boost | $[0.85, 1.15]$ |

## Stage 1: Task-Weighted Sum

$$S_{\text{task}} = \beta_{\text{CTR}} \cdot p_{\text{CTR}} + \beta_{\text{CVR}} \cdot p_{\text{CVR}} + \beta_{\text{NBA}} \cdot p_{\text{NBA}} + \beta_{\text{LTV}} \cdot p_{\text{LTV}} \tag{12}$$

Default weights: CVR=0.4 (highest, conversion directly drives revenue), CTR=0.3, NBA=0.2, LTV=0.1. Since $\sum \beta_i = 1$ and all $p_i \in [0, 1]$, this is a convex combination in the WSM (Weighted Sum Model, Fishburn 1967) framework, guaranteeing $S_{\text{task}} \in [0, 1]$.

## Stage 2: Financial DNA Modifier

Based on Friedman's Permanent Income Hypothesis (1957): consumers make spending decisions based on their permanent (stable) income rather than transient income fluctuations.

$$W_{\text{DNA}} = \begin{cases} 1.2 \text{ if CV} < 0.2 \quad \text{(Permanent -- stable income)} \\ 1.0 \text{ if } 0.2 \leq \text{CV} < 0.5 \quad \text{(Mixed)} \\ 0.8 \text{ if CV} \geq 0.5 \quad \text{(Transitory -- unstable income)} \end{cases} \tag{13}$$

Here CV $= \sigma_{\text{income}}/\mu_{\text{income}}$ (coefficient of variation, dimensionless). Customers with stable income are well-suited for long-term financial products (annuities, fixed-term deposits), so the recommendation score is boosted by 20%.

## Stage 3: TDA Behavioral Velocity

$$V_{\text{TDA}} = 1.0 + \gamma_{\text{flare}} \cdot \mathbb{1}[\text{flare}_{\text{detected}}] \tag{14}$$

$\gamma_{\text{flare}} = 0.15$. TDA flare detection indicates acceleration of behavioral change, boosting the score by up to 15%.

## Stage 4: Risk Penalty

$$R = 0.2 \cdot I_{\text{limit}} + 0.3 \cdot I_{\text{fatigue}} + 0.5 \cdot I_{\text{churn}} \tag{15}$$

**Rationale for asymmetric weights – irreversibility:**

- Credit limit exhaustion ($\lambda_1 = 0.2$): reversible (limit restored upon repayment)
- Message fatigue ($\lambda_2 = 0.3$): partially reversible (recovers over time)
- Customer churn ($\lambda_3 = 0.5$): nearly irreversible (re-acquisition cost is 5–7x that of new acquisition)

$(1 - R)$ exercises multiplicative veto power. As $R \to 1$, the score converges to 0 regardless of other factors. In log space, $\ln(1 - R) \to -\infty$ as $R \to 1$.

## Fatigue Decay

$$f(n) = e^{-\lambda n} \tag{16}$$

Exponential decay implements **constant fractional decay**: $f(n+1)/f(n) = e^{-\lambda}$ (constant ratio).

Half-life: $n_{1/2} = \ln 2/\lambda$. App Push ($\lambda = 0.4$): half-life $\approx 1.73$ exposures. Email ($\lambda = 0.15$): half-life $\approx 4.62$ exposures.

## Confidence Formula

$$\text{confidence} = |p - 0.5| \times 2 \tag{17}$$

Normalizes the distance from the decision boundary (0.5) to $[0, 1]$. Predictions with low confidence are deprioritized in the recommendation quality filter.

# Recommendation Reason Generation

## 2-Layer Architecture (v3.0.0)

Design philosophy: "Equal explanation for all customers; LLM-enhanced explanation for context-rich customers."

| Layer | Target | Method | LLM Calls | Throughput |
|---|---|---|---|---|
| L1 | All 12M customers | Template via 3-agent pipeline (FactExtractor → InterpretationRegistry → TemplateEngine → SelfChecker) | 0 | ∼20 min |
| L2a | ∼500K/week | LLM rewrite (Bedrock Claude Haiku, 3-layer safety gate) | 1 | ∼1.0 sec/record |
| L2b | ∼67K sampled | Quality validation (Bedrock Claude Haiku; factuality, relevance, naturalness) | 1 | – |

### 3-Agent Reason Pipeline (L1)

L1 reason generation runs a deterministic 4-stage agent pipeline — zero LLM calls:

1. **FactExtractor:** extracts customer-level narrative facts from feature values via YAML-configured Python expressions (e.g., "deposit-focused portfolio", "risk-averse tendency"). Sandboxed, fully deterministic.
2. **InterpretationRegistry:** maps IG attribution top-5 features to financial language via pre-registered 3-tuple enrichments (feature → display_name, direction, explanation). Falls back to `ReverseMapper` (Level RM) if no entry found.
3. **TemplateEngine:** selects template variant via $\text{hash}(\text{customer}_{\text{id}} : \text{category}) \bmod 5$ and injects FactExtractor output + IG interpretations.
4. **SelfChecker:** 3-stage verification gate — (1) compliance: rule match against prohibited patterns (comparative claims, guaranteed-return statements, etc.), (2) injection: prompt-injection pattern detection, (3) factuality: LLM-based factual consistency check. Only reasons passing all three stages are released.

**Regulatory basis:** Article 19 of the Financial Consumer Protection Act requires equal duty of explanation for all customers. L1 fulfills this requirement by providing zero-cost templates to all 12M customers.

**Cost efficiency:** Full LLM processing would consume ∼1,000 GPU-hours. The 2-Layer design achieves the same coverage in ∼162 GPU-hours.

# L1 Template Generation

### Structure

6 categories × 5 variants = 30 templates.

**Deterministic variant selection:**

$$\text{variant}_{\text{index}} = \text{hash}(\text{customer}_{\text{id}} : \text{category}) \bmod 5 \tag{18}$$

The same customer always receives the same variant (consistency + audit reproducibility).

### Segment Awareness

- **WARMSTART**: Reasons based on IG Top-3 reverse mapping
- **COLDSTART**: Reasons based on popularity + benefits
- **ANONYMOUS**: Reasons based on general popularity

After a rule-based compliance check, an AI-generated disclosure notice is automatically appended.

# L2a LLM Rewrite

Priority queue: rich first, moderate second, sparse excluded. Rewriting is applied after passing through the 3-Layer Safety Gate. **AWS deployment:** Bedrock Claude Haiku ($0.25/1M input tokens; VPC PrivateLink; no data transmitted to Anthropic for training). **On-premises:** vLLM Qwen3-8B-AWQ on RTX 4070 (12GB VRAM).

### 4-Layer Prompt Structure

1. **System prompt**: Role definition (financial recommendation reasoning expert) + prohibition rules under the Financial Consumer Protection Act
2. **Few-shot examples**: Tone and format guide per segment
3. **Context injection**: Customer features, IG attributions, consultation history → natural language conversion
4. **Output format**: JSON schema (`{"reasons": [...], "summary": "..."}`)

### Decoding Strategy

- Reason generation: $\tau = 0.3$ (fact preservation + slight diversity)
- Critique: $\tau = 0.1$ (near-deterministic, consistent quality assessment)
- L2a Rewrite: $\tau = 0.3$ (preserve original facts, polish phrasing)

# Self-Critique Verdict

$$\text{verdict} = \begin{cases} \text{pass} & \text{if } f \geq 0.8 \text{ and } c \geq 1.0 \\ \text{revise} & \text{if } f \geq 0.5 \text{ and } c \geq 1.0 \\ \text{reject} & \text{otherwise} \end{cases} \tag{19}$$

- $f$: factuality score (continuous)
- $c$: compliance score (binary: 1.0 = no violation)

**Compliance takes absolute priority:** any regulatory violation ($c < 1.0$) results in immediate rejection regardless of factuality. **At most one revision:** if the verdict is still "revise" after revision, fall back to a safe template (prevents infinite loops; maximum 3 LLM calls).

## L2b 3-Axis Quality Validation

$$\text{verdict} = \begin{cases} \text{pass} & \text{if } f \geq 0.7 \text{ and } r \geq 0.7 \text{ and } n \geq 0.7 \\ \text{needs}_{\text{improvement}} & \text{if any score} \in [0.5, 0.7) \\ \text{fail} & \text{if any score} < 0.5 \end{cases} \tag{20}$$

- $f$: factuality, $r$: relevance, $n$: naturalness
- Threshold 0.7 (lower than Self-Critique's 0.8): L2b is post-hoc monitoring, not a real-time gatekeeper

# Grounding + Feature Reverse Mapping

## Core Problem

PLE-adaTT consumes a feature vector (734D full-bank design; 1205D in the current Santander 17-group implementation) and outputs probability scores, but cannot explain *why* a given recommendation was made. Articles 31 and 34 of the AI Basic Act, and Article 19 of the Financial Consumer Protection Act, require meaningful explanations.

## Grounding Function

$$f : \mathbb{R}^{644} \times \mathcal{I} \to \mathcal{L} \tag{21}$$

Here $\mathbb{R}^{644}$ is the normalized feature vector space, $\mathcal{I}$ is the IG attribution information, and $\mathcal{L}$ is the natural language explanation space.

## 734D Feature Vector Structure

| Range | Dimensions | Content |
|---|---|---|
| profile | 0–238 | Demographics (100D) + RFM (50D) + Financial Summary (88D) |
| multi_source | 238–329 | Transaction stats (40D) + Behavioral patterns (51D) |
| extended_source | 329–413 | Insurance, consultation, STT, campaign, overseas, open banking |
| domain | 413–572 | TDA (70D) + GMM (22D) + Mamba (50D) + Economics (17D) |
| model_derived | 572–599 | HMM summary, Bandit/MAB, LNN |
| multidisciplinary | 599–623 | Conversion dynamics, adoption dynamics, cross patterns, routine analysis |
| merchant_hierarchy | 623–644 | MCC levels, brand embeddings, statistics, radius |

Total: 644D normalized + 90D raw power-law = 734D model input.

## Integrated Gradients Attribution

$$\mathrm{IG}_i(\boldsymbol{x}) = (x_i - x_i') \times \int_0^1 \frac{\partial F(\boldsymbol{x}' + \alpha(\boldsymbol{x} - \boldsymbol{x}'))}{\partial x_i} d\alpha \tag{22}$$

**Why IG is more suitable than SHAP:**

- SHAP: requires evaluating $2^{734}$ subsets at full-bank scale (infeasible)
- IG: computed in linear time via a 50-step trapezoidal approximation

- **Completeness axiom:** $\sum_i \text{IG}_i(\boldsymbol{x}) = F(\boldsymbol{x}) - F(\boldsymbol{x}')$ (guaranteed by the gradient theorem in vector calculus)
- **Baseline:** zero vector (corresponds to "average customer" in normalized feature space)

## Reverse Mapping Architecture

$$\text{ReverseMap} : \left(\boldsymbol{x} \in \mathbb{R}^d, \boldsymbol{a} \in \mathbb{R}^d\right) \to \{(r_k, s_k, t_k)\}_{k=1}^{K} \tag{23}$$

- $\boldsymbol{x}$: feature vector, $\boldsymbol{a}$: IG attribution vector
- $r_k$: feature range name, $s_k$: summary score, $t_k$: financial language text

**Sub-range slicing:** $t_k = \mathcal{M}_k(g(\boldsymbol{x}[s_k : e_k]))$ where $g$ is an aggregation function (mean, argmax, threshold comparison), and $\mathcal{M}_k$ is a domain-expert-designed mapping dictionary (numerical → text).

## Module Composition

- **FeatureReverseMapper:** 644D vector → financial language text (hierarchical range slicing)
- **MultidisciplinaryInterpreter:** 24D multidisciplinary features → business interpretation (4 sub-domains)
- **LanceContextVectorStore:** LanceDB-based customer context storage/retrieval (768D embedding, L2 distance)
- **ContextAssemblyAgent:** IG-based tool selection + multi-source context assembly
- **ConsultationContextExtractor:** STT consultation history extraction + summarization

## Trust Loop

Model prediction → IG attribution → reverse mapping → context assembly → LLM reason generation → agent delivery → customer persuasion → conversion/feedback → model improvement.

Without reverse mapping and context assembly, an interpretability gap arises between model predictions and agent delivery, breaking this Trust Loop.

## Triple Grounding

1. **Feature Grounding:** inject IG Top-5 attributions into the prompt → LLM generates reasons grounded in the model's actual decision basis
2. **Customer Grounding:** inject segment, transaction patterns, and consultation history → hallucination suppression
3. **Regulatory Grounding:** system prompt prohibition rules + Rule-based Self-Critique → compliance enforcement

# Safety Gate

## Multi-Layer Defense Architecture

A 6-layer safety mechanism for compliance with the Financial Consumer Protection Act and the AI Basic Act.

| Layer | Mechanism | Target Regulation |
|---|---|---|
| 1 | System prompt: immutable regulatory prohibition rules | AI Basic Act Art. 31 & 34 |
| 2 | Self-Critique: real-time gatekeeper ($f \geq 0.8$, $c = 1.0$) | KFCPA Art. 19 & 21 |
| 3 | 3-Layer Safety Gate: prompt injection detection + factuality + regulatory compliance | KFCPA Art. 22 |
| 4 | L2b Quality Validation: post-hoc monitoring 3-axis assessment | Internal quality standards |
| 5 | AI Security Checker: injection detection + compliance verification | AI Basic Act Art. 34 |
| 6 | Audit Archiver: immutable Parquet records (DuckDB-based retrieval) | FSS post-inspection |

## Applicable Regulations

- **AI Basic Act Art. 31** (AI usage disclosure): AI-generated disclosure notice automatically appended to all recommendation reasons
- **AI Basic Act Art. 34** (risk management): Safety Gate + audit trail
- **Financial Consumer Protection Act Art. 19** (suitability principle + duty of explanation): L1 guarantees full coverage
- **Financial Consumer Protection Act Art. 21** (advertising regulations): prohibited pattern detection
- **Financial Consumer Protection Act Art. 22** (prohibition of unfair practices): compliance score verification

## Audit Archiving

`RecommendationAuditArchiver` persistently stores all recommendation records as Parquet.

- IG attribution scores, L1 reasons, L2a rewrite results, L2b validation results, processing time
- Efficient retroactive retrieval via DuckDB
- Full decision path tracing for individual recommendation records during FSS post-inspection

# Serving Architecture

## End-to-End Pipeline

```
PLE-adaTT Teacher (training)
  |-> Knowledge Distillation (T=5, alpha=0.3) + Adaptive Threshold (2x random floor)
    |-> LGBM Student (per-task, 200D features) + Platt Calibration (probability-critical
tasks)
      |-> ONNX Export (ZipMap removal)
        |-> Lambda FallbackRouter
            |-> Layer 1: LGBM ONNX (primary)
            |-> Layer 2: PLE SageMaker Endpoint (failover)
            |-> Layer 3: Rule Engine (12 tasks, Financial DNA routing)
          |-> FD-TVS Scoring (4-Stage)
            |-> Feature Grounding (IG -> Reverse Mapping -> Context Assembly)
              |-> Recommendation Reason
                  L1: FactExtractor -> InterpretationRegistry -> TemplateEngine -> SelfChecker
                  L2a: Bedrock Claude Haiku (rewrite)
                  L2b: Bedrock Claude Haiku (validation)
                |-> Audit Archive (Parquet, ComplianceAuditStore)
```

## LGBM → ONNX Conversion

### ZipMap Removal (Required)

LightGBM's ONNX conversion adds a ZipMap operator, producing dictionary output. Since Triton supports only tensor output, the ZipMap node must be bypassed and removed from the ONNX graph.

### Conversion Specifications

- Opset 13: full support for LightGBM operators
- 2-stage validation: (1) `onnx.checker.check_model` specification conformance, (2) dummy inference numerical equivalence test

## Triton Inference Server Configuration

### Model Deployment

15 ONNX models (per-task) + 1 preprocessor + 1 postprocessor + 15 ensemble schedulers = **32 model configurations**.

### Dynamic Batching

- Preferred batch sizes: [256, 512, 1024]
- Max queue delay: 100$\mu$s
- Preprocessor: CPU × 4 instances (CPU-bound JSON parsing)
- ONNX models: GPU × 2 instances

### Batch + Real-Time Hybrid

Daily batch processing computes baseline scores for all customers; when a real-time transaction occurs, FD-TVS scores are immediately recomputed incorporating real-time features from the Redis cache. Triton Dynamic Batching queues individual real-time requests into micro-batches, maintaining GPU utilization.

## Training-Serving Skew Prevention

**Feature Serving Spec** bridges training and serving.

- `feature_selector` outputs `selected_features_{task}.json` during training
- `FeatureServingSpec` loads this at deployment time to guarantee identical feature ordering
- The 7 mandatory features (TDA, Economics, FinEng) are always included

## Calibration Considerations

FD-TVS Stage 1 requires all task probabilities to lie on a common scale $[0, 1]$. Probability-critical tasks (churn, product_stability, cross_sell_count) apply Platt scaling post-distillation (see §1 Calibration). Other tasks use Temperature Scaling (Guo et al., ICML 2017) as a lightweight alternative.

## 3-Layer Fallback Architecture

The Lambda serving layer implements a `FallbackRouter` that selects among three layers based on availability and confidence:

| Layer | Mechanism | Produces |
|---|---|---|
| Layer 1: Distilled LGBM | ONNX model via Lambda; Platt-calibrated per task | `scores`, `contributing_features` (IG top-5) |
| Layer 2: Direct PLE | SageMaker Endpoint; activated if LGBM SKIP or Lambda cold-start failure | `scores`, `contributing_features` (IG top-5) |
| Layer 3: Rule Engine | Python rule set: 12 task-specific rules + Financial DNA feature routing | `scores` (heuristic), `contributing_features` (rule-based) |

All three layers produce `contributing_features` to maintain explanation compliance under AI Basic Act Art. 34 and Financial Consumer Protection Act Art. 19, even in degraded-mode serving.

### Rule Engine Design

The rule engine implements 12 task-specific rules organized around Financial DNA feature routing:

- **DNA routing:** permanent income customers (CV $< 0.2$) $\rightarrow$ long-term product rules; transitory customers (CV $\geq 0.5$) $\rightarrow$ short-term, low-commitment rules
- **Task rules:** each of the 12 tasks has a heuristic score function based on 3–5 interpretable features (e.g., churn rule uses recency + frequency + support call count)
- **contributing_features:** the rule selects the top-3 features that triggered the rule and returns them as `contributing_features` for the reason generation pipeline

# LLM Distillation: Gemini Teacher → Qwen Student (QLoRA)

### Distinguishing the Two Types of Distillation

| Aspect | Predictive Model Distillation | LLM Distillation |
|---|---|---|
| Purpose | Prediction accuracy | Text generation quality |
| Teacher | PLE-Cluster-adaTT | Google Gemini |
| Student | LightGBM | Qwen3-8B (on-prem) |
| Transfer Target | Soft labels (logits/probs) | Text output (recommendation reasons) |
| Loss Function | KL Divergence + CE | Cross-Entropy (SFT) |
| Training Method | Soft label learning | QLoRA fine-tuning |

**AWS serving note:** In production AWS Lambda, L2a rewrite and L2b validation use **Bedrock Claude Haiku** (not Qwen3-8B). Qwen3-8B QLoRA is the on-premises alternative. Both share the same PromptSanitizer and 3-Layer Safety Gate.

### QLoRA: Mathematical Foundation of LoRA

$$W' = W_0 + \Delta W = W_0 + BA \tag{24}$$

- $W_0 \in \mathbb{R}^{d \times k}$: original pre-trained weights (frozen)
- $B \in \mathbb{R}^{d \times r}$: Down-projection (trainable)
- $A \in \mathbb{R}^{r \times k}$: Up-projection (trainable)
- $r \ll \min(d, k)$: Rank ($r = 16$ corresponds to 0.78% of the original)

Compression ratio: $r \times (d + k)/(d \times k)$. Qwen3-8B ($d = k = 4096$, $r = 16$): $131,072/16,777,216 \approx 0.78\%$.

### NF4 Quantization

Distribution-aware quantization that places quantization levels at quantiles of the standard normal distribution.

$$q_i = \Phi^{-1}\big(i/\big(2^k + 1\big)\big) \tag{25}$$

Each level covers equal probability mass, satisfying the Lloyd-Max optimality condition.

### QLoRA Memory Analysis

- Full FT (FP16): weights 16GB + optimizer 32GB + gradients 16GB = 64GB+ (infeasible on RTX 4070)
- QLoRA: base model 4GB (NF4) + LoRA adapter ~40MB = **trainable in 6GB**

### Self-Consistency Training Data Filtering

$$\text{consistency}(s_1, s_2, s_3) = \min_{i \neq j} \text{BERTScore}\big(s_i, s_j\big) \tag{26}$$

Three outputs are generated from the Gemini Teacher for the same input, and the minimum pairwise BERTScore is used as a conservative consistency measure. Only consistent outputs are included in training data to filter out Teacher hallucinations.

# Cost Efficiency and Operational Summary

## Cost Efficiency Summary

| Component | Design Choice | Effect |
|---|---|---|
| 2-Layer Reason Generation | L1 Template + L2 LLM | 162 vs 1,000 GPU-hours |
| LGBM Student | CPU inference | 1/10 cost vs GPU Teacher |
| QLoRA | NF4 + LoRA | 6GB vs 64GB+ training |
| Triton Dynamic Batching | Micro-batch queuing | Maximizes GPU utilization |

## Cross-Cutting Concerns

**Financial Domain Specialization**

- **Distillation:** TDA, Economics, and FinEng mandatory features are preserved regardless of IG importance
- **Scoring:** DNA modifier based on Friedman's Permanent Income Hypothesis; asymmetric risk weighting based on irreversibility
- **Grounding:** financial language translation via domain-expert-designed mapping dictionaries
- **Reason generation:** regulation-first design (compliance takes precedence over quality)
- **LLM distillation:** financial terminology, compliance-aware tone, and product-specific knowledge transfer

**End-to-End Pipeline Coherence**

Each stage of distillation → serving → scoring → grounding → reason generation → audit maintains end-to-end coherence through a shared contract: the Feature Serving Spec, IG attributions, and reverse mapping dictionaries. If this contract is broken at any stage, errors propagate to all downstream stages, making per-stage validation essential.

# Ops/Audit Agent Integration

Recommendation reason quality is monitored under AuditAgent's AV3 viewpoint via a 3-Tier system:

- **Tier 1** (exhaustive): SelfChecker pass/revise/reject rate trends
- **Tier 2** (sampled): stratified sampling across 27 strata → GroundingValidator (reason↔IG alignment)
- **Tier 3** (expert): 50–100 monthly manual reviews → feedback loop

InterpretationRegistry → 3-tuple enrichment → TemplateEngine integration is complete, embedding IG interpretations into L1 reasons. ReverseMapper is integrated as Level RM fallback in InterpretationRegistry, expanding feature interpretation coverage.

## Fact Extraction Layer (New, 2026-04)

`FactExtractor` extracts **customer narrative facts** from feature values. While `InterpretationRegistry` provides feature-level interpretation, `FactExtractor` adds customer-level profiles ("deposit-focused portfolio", "risk-averse tendency").

### Rule-Based, Zero LLM

Python expressions defined in YAML config (`configs/financial/fact_extraction.yaml`) are evaluated in a sandboxed environment (`__builtins__` disabled). Zero LLM calls, fully deterministic. Facts are extracted at Phase 0 batch time and stored in `ContextVectorStore`; at serving time, they are merely retrieved and injected into the L2a prompt.

### L2a Prompt Integration

Both `AsyncReasonOrchestrator.generate_l1()` and `get_best_reason()` retrieve `context_store.get_context(customer_id).get("customer_facts", [])` and include them as `context["customer_facts"]` in the SQS message body. `_build_llm_prompt()` injects them as a "## Customer Facts" section in the final prompt.

Detailed design: Design Document 11 (`docs/design/11_ops_audit_agent.typ`)